

Transport Layer

The Transport Layer is the OSI Layer 4 responsible for providing reliable, efficient, and ordered end-to-end communication between applications on different hosts.

- Operates at Layer 4 (Transport Layer) of the OSI model
- Enables process-to-process communication using port numbers
- Ensures reliability through error control, sequencing, and retransmission
- Supports flow control to prevent receiver overload
- Uses protocols like TCP, and UDP for different application needs

Functions of Transport Layer

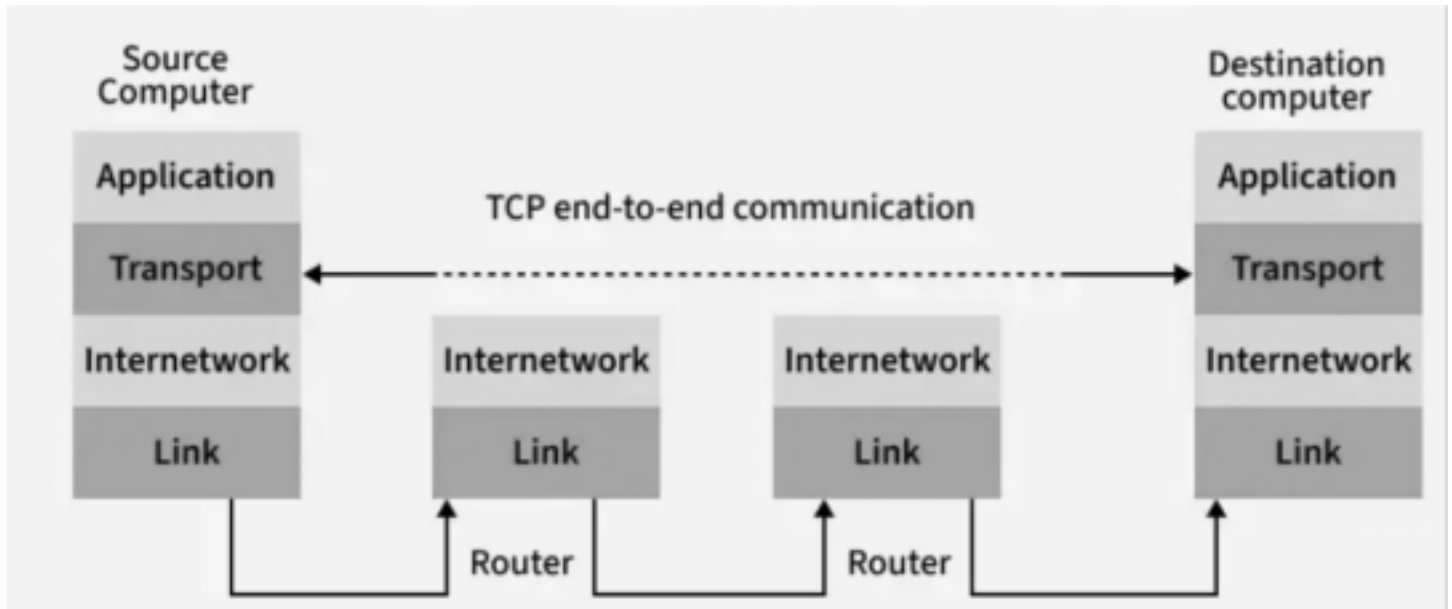
The Transport Layer is responsible for end-to-end communication of data packets. It provides a number of important functions that are responsible for reliable, efficient, and organized data transfer between host systems in a networked environment.

The primary functions of the Transport Layer are:

- Provides end-to-end data delivery
- Ensures reliable and ordered data transfer
- Performs error detection and recovery
- Controls data flow to prevent receiver overload
- Supports multiplexing and demultiplexing using port numbers

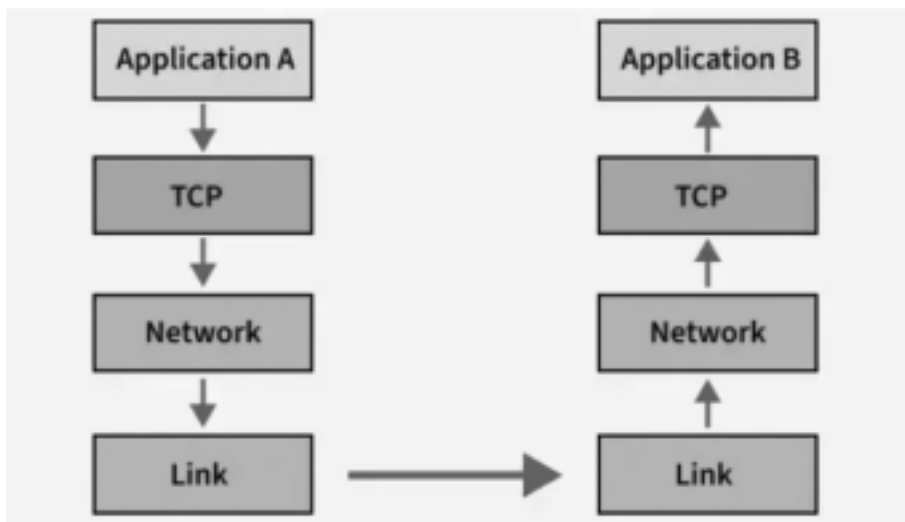
1. End-to-end Connection between Hosts

The transport layer is also responsible for creating the end-to-end Connection between hosts, for which it mainly uses TCP and UDP. TCP is a secure, connection-oriented protocol that uses a handshake protocol to establish a robust connection between two end hosts. TCP ensures the reliable delivery of messages and is used in various applications. UDP, on the other hand, is a stateless and unreliable protocol that ensures best-effort delivery. It is suitable for applications that have little concern with flow or error control and requires sending the bulk of data like video conferencing. It is often used in multicasting protocols.



2.Flow Control

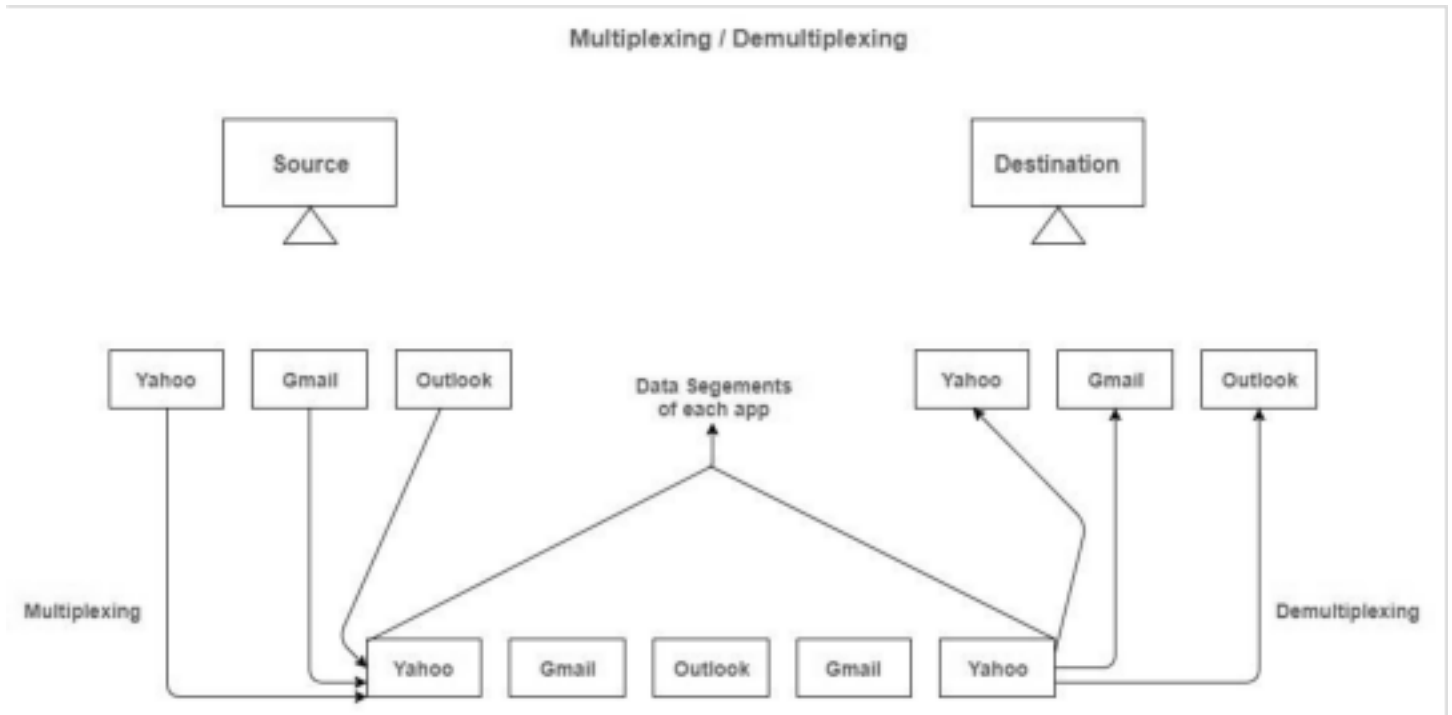
The transport layer provides a flow control mechanism between the adjacent layers of the TCP/IP model. TCP also prevents data loss due to a fast sender and slow receiver by imposing some flow control techniques. It uses the method of sliding window protocol which is accomplished by the receiver by sending a window back to the sender informing the size of data it can receive.



3. Multiplexing and Demultiplexing

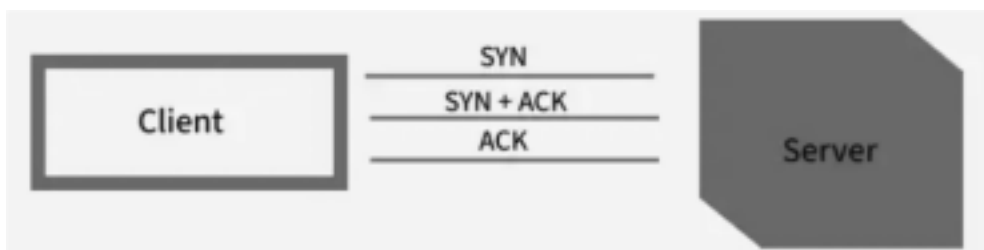
Multiplexing(many to one) is when data is acquired from several processes from the sender and merged into one packet along with headers and sent as a single packet. Multiplexing allows the simultaneous use of different processes over a network that is running on a host. The processes are differentiated by their port numbers. Similarly, Demultiplexing(one to many) is required at the receiver side when the message is

distributed into different processes. Transport receives the segments of data from the network layer distributes and delivers it to the appropriate process running on the receiver's machine.



4.Connection Establishment

When two devices in a network wants to establish a connection using TCP, it is done through **3-Way Handshake Process**.

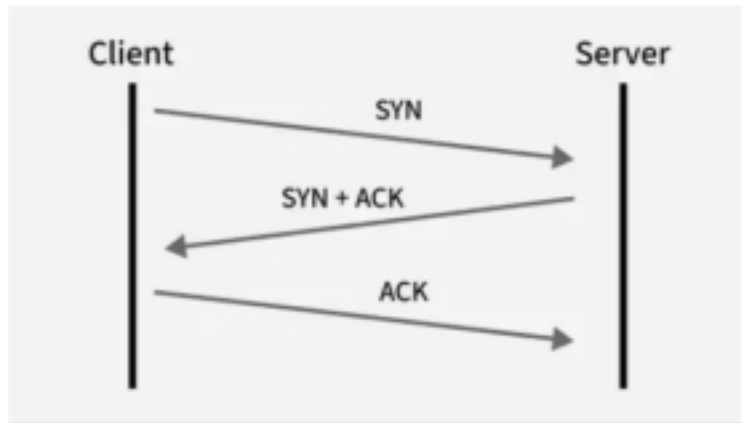


- The first computer connects to the second computer by sending a SYN packet to a specified port number.
- If the second computer is listening, it will respond with a SYN/ACK.
- When the first computer receives the SYN/ACK, it replies with an ACK packet.
- After this, the two devices can communicate normally.

5. Connection Termination

In a TCP connection, we have two types of termination mechanisms:

1. In the Graceful connection release, the connection is open until both parties have closed their sides of the connection.
2. In an Abrupt connection release, either one TCP entity is forced to close the connection or one user closes both directions of data transfer.



6. Reliable Data Delivery

The transport layer checks for errors in the messages coming from the application layer by using error detection codes, and computing checksums, it checks whether the received data is not corrupted and uses the ACK and NACK services to inform the sender if the data has arrived or not and checks for the integrity of data.

TCP 3-Way Handshake Process

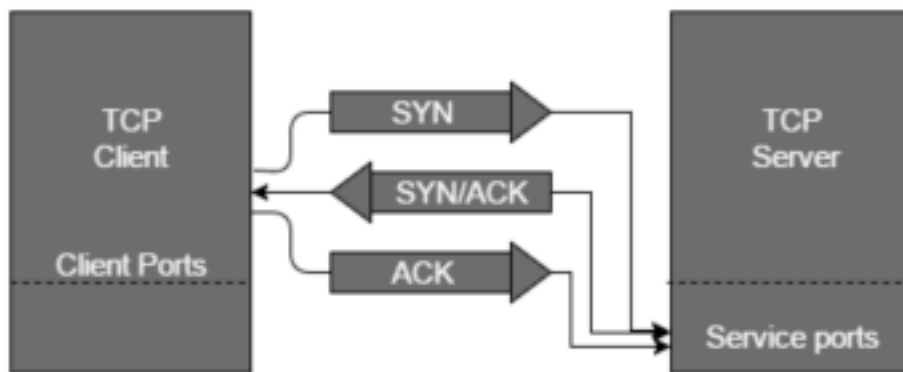
The TCP 3-Way Handshake is a process used by the **Transmission Control Protocol (TCP)** to establish a reliable connection between a client and a server before data transfer. It ensures that both sides are synchronized and ready to communicate.

TCP 3-way Handshake Process

Communication over the internet follows the TCP/IP model. Applications like web browsers use the Application Layer, and their data is passed to the Transport Layer,

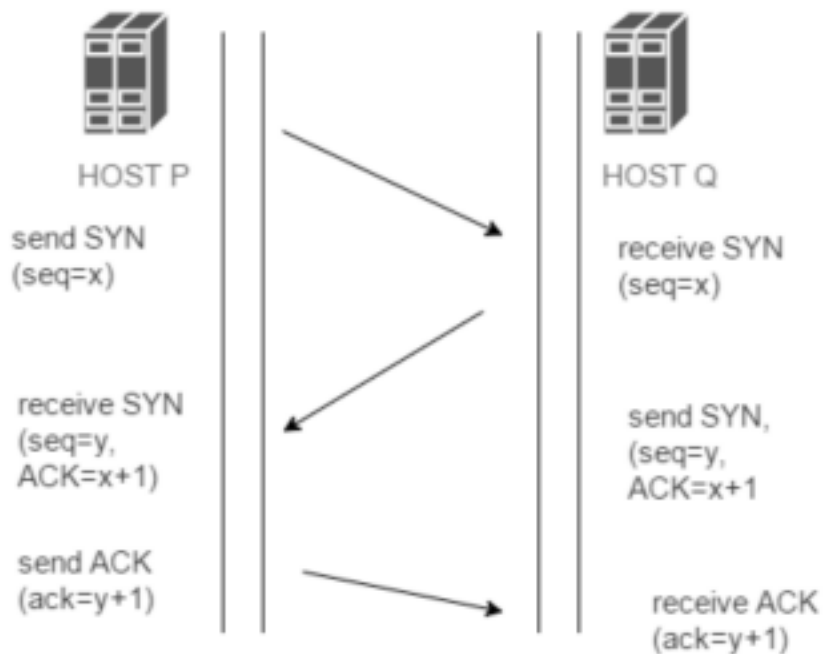
where TCP and UDP work.

- TCP is widely used because it provides reliable communication. (e.g., money transfer)
- UDP is faster but unreliable (e.g., used in DNS lookups, video call, voice call).



TCP ensures reliability using Positive Acknowledgement with Retransmission (PAR):

- Each segment sent must be acknowledged.
- If a segment is lost or corrupted (detected using the checksum), the receiver discards it, and the sender retransmits.



- **Step 1 (SYN):** In the first step, the client wants to establish a connection with a server, so it sends a segment with SYN(Synchronize Sequence Number) which informs the server that the client is likely to start communication and with what sequence number it starts segments with
- **Step 2 (SYN + ACK):** Server responds to the client request with SYN-ACK signal bits set. Acknowledgement(ACK) signifies the response of the segment it received and SYN signifies with what sequence number it is likely to start the segments with
- **Step 3 (ACK):** In the final part client acknowledges the response of the server and they both establish a reliable connection with which they will start the actual data transfer.

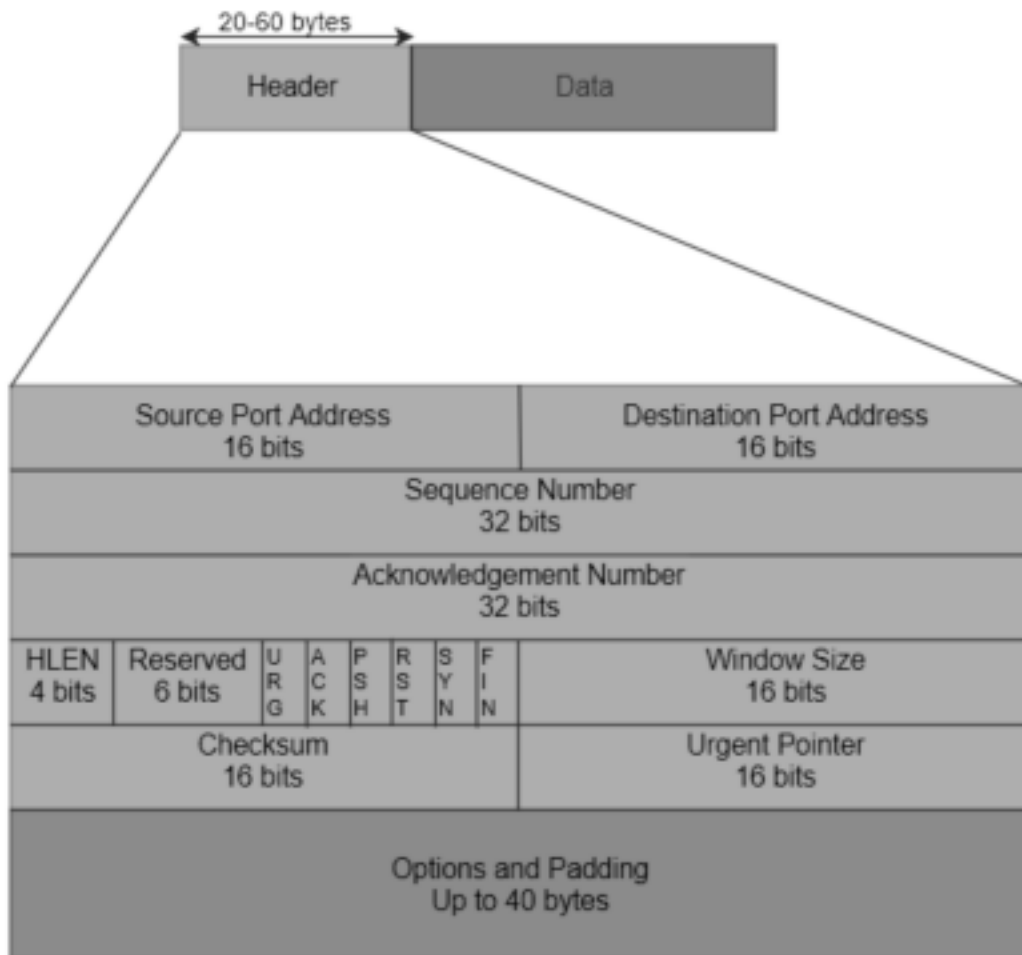
Why not just two steps?

With only SYN and SYN-ACK, neither the client nor the server can be sure that the other has successfully received the messages. The third ACK provides mutual confirmation, ensuring both sides are ready before data transfer begins.

Transmission Control Protocol (TCP)

Segment Structure

A TCP segment consists of data bytes to be sent and a header that is added to the data by TCP as shown:



The header of a TCP segment can range from 20-60 bytes. 40 bytes are for options. If there are no options, a header is 20 bytes else it can be of upmost 60 bytes. Header

fields:

1. **Source Port / Destination Port (16 bits each):** Identify sending and receiving applications. (ex: HTTP → Port **80**)
2. **Sequence Number (32 bits):** Position of the first byte in this segment, used for ordering. (ex: Segment 1 → Bytes **1–1000**)
3. **Acknowledgement Number (32 bits):** Next byte expected by the receiver (confirms data received). (ex: ACK = **1001**)
4. **Header Length (HLEN):** Size of the header (5–15 words, i.e., 20–60 bytes).
5. **Control Flags (1 bit each):**
 - **URG:** Urgent data
 - **ACK:** Acknowledgement valid
 - **PSH:** Push data immediately
 - **RST:** Reset connection
 - **SYN:** Synchronize sequence numbers
 - **FIN:** Terminate connection
6. **Window Size:** Receiver's buffer size (for flow control).
7. **Checksum:** Error detection (mandatory).
8. **Urgent Pointer:** Position of urgent data (if URG flag is set).

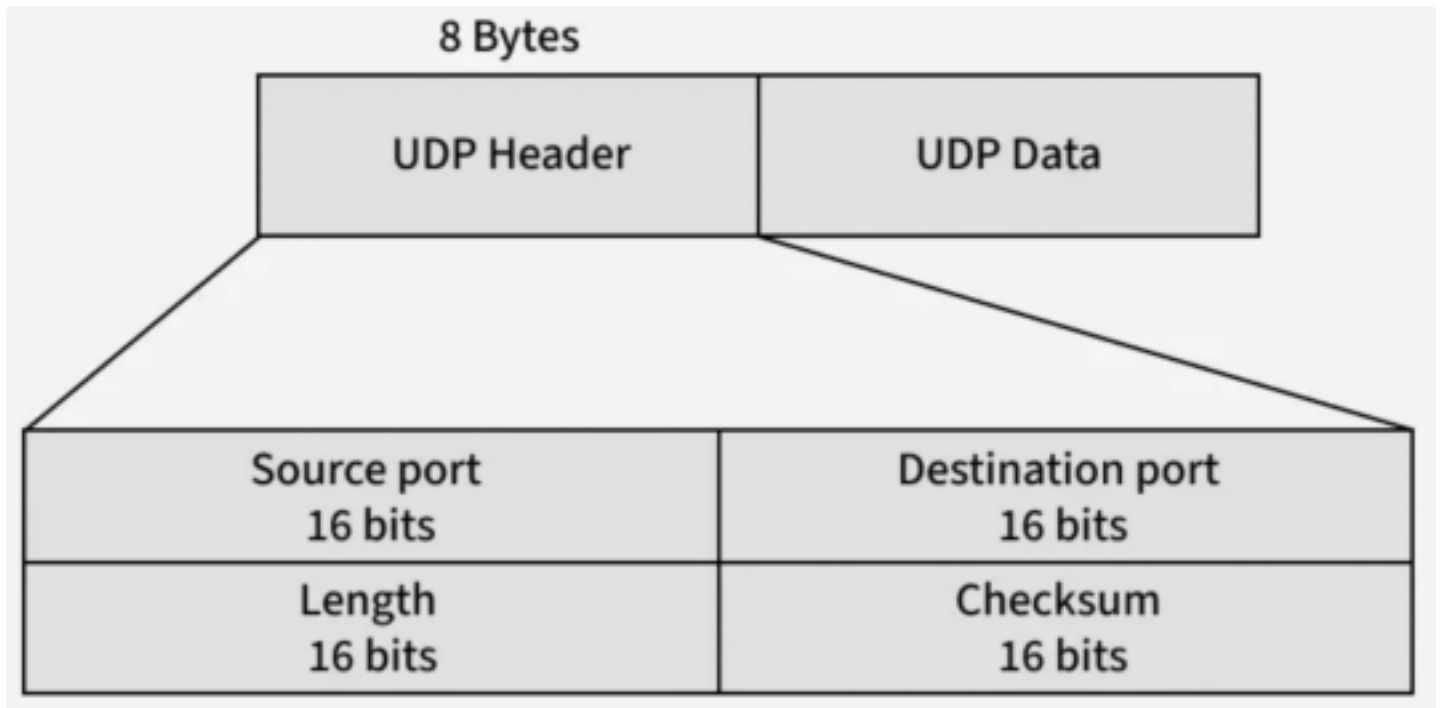
User Datagram Protocol - UDP



User Datagram Protocol (UDP) is a Transport Layer protocol of the Internet Protocol (IP) that provides fast, connectionless, and lightweight communication between processes. It does not guarantee delivery, order, or error checking, making it suitable for real-time and time-sensitive applications such as video streaming, DNS, and VoIP.

UDP Header

UDP header is 8 bytes long, followed by the data payload. It contains all the essential information needed for transmission. Each port number field is 16 bits, giving a range from 0 to 65535, where port 0 is reserved. Port numbers are used to identify and separate different user requests or processes.



UDP Header fields description

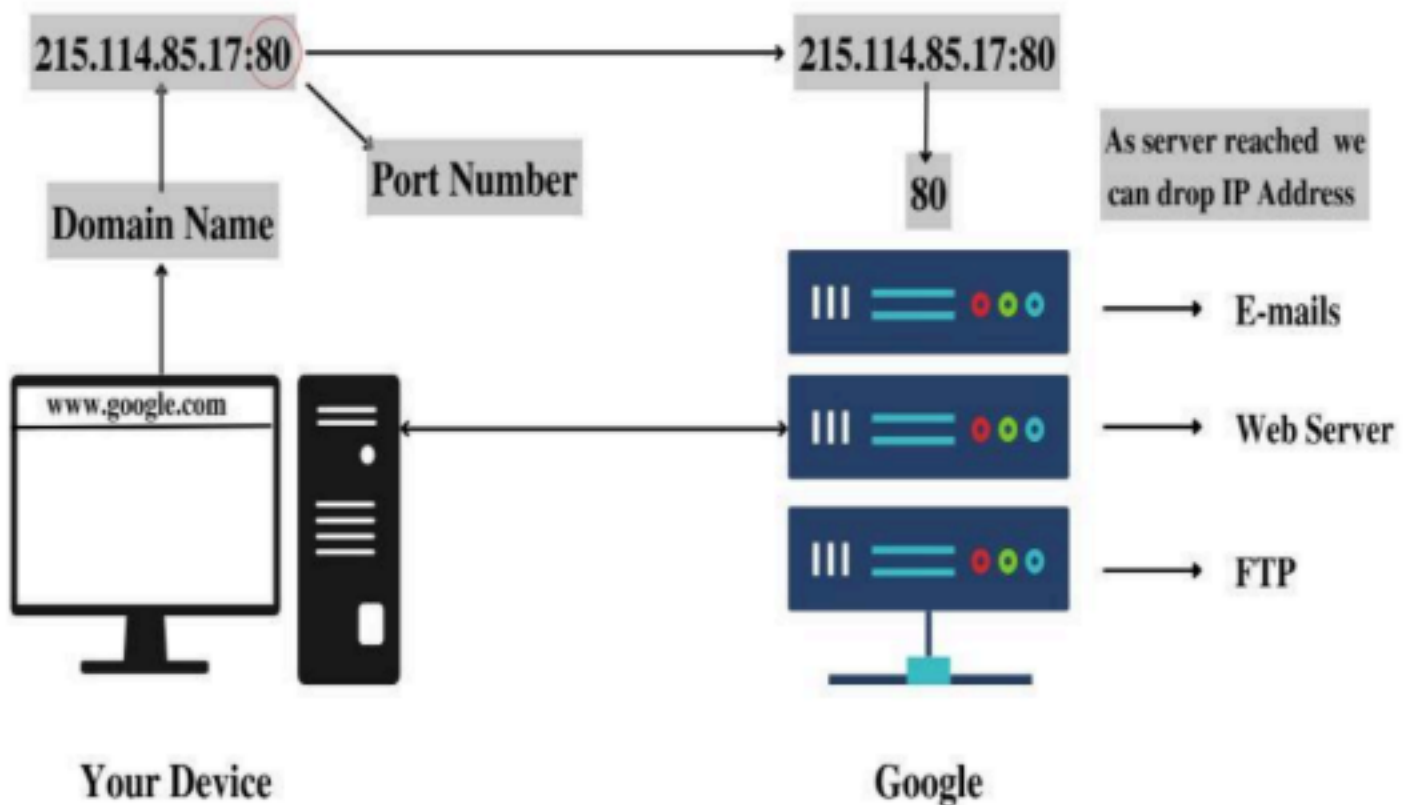
Field	Size	Description
Source Port	16 bits	Identifies the sender's port number.
Destination Port	16 bits	Identifies the receiver's port number.
Length	16 bits	Specifies the total length of UDP header and data.
Checksum	16 bits	Used for error detection (optional in IPv4, mandatory in IPv6).

Ports in Transport layer

A port is a logical identifier used to distinguish different applications or services on a device, allowing network traffic to reach the correct program.

- Ports work at the Transport layer, using TCP and UDP to send and receive data between devices.

- They enable multiple applications to use the network simultaneously without interference.
- Ports help the operating system route incoming data to the appropriate application.



How Ports Work

When a device communicates over a network, data packets are sent to its IP address. Each packet also includes a port number, which tells the operating system which application or service should receive that data.

- **Port numbers identify services:** Each application listens on a specific port, so incoming data reaches the correct program.
- **IP + Port + Protocol = Socket:** This combination ensures that network traffic is

delivered precisely to the right process on the right device.

- **TCP and UDP protocols use ports differently:** TCP ensures reliable delivery, while UDP is faster but without guaranteed delivery.

Port Number Range

A port number is a 16-bit number used to identify network services on a device. Since it is 16 bits, it can have values from 0 to 65535, giving enough unique numbers to support many applications running simultaneously.

- **0–65535:** Total possible port numbers.
- Lower numbers are usually reserved for standard services (like HTTP, FTP, SSH).
- Higher numbers are often used temporarily for client connections or custom applications.
- Port numbers help organize and manage network traffic efficiently, preventing conflicts between different services.

Types of Ports

Ports are mainly classified into the following types:

1. Physical Ports

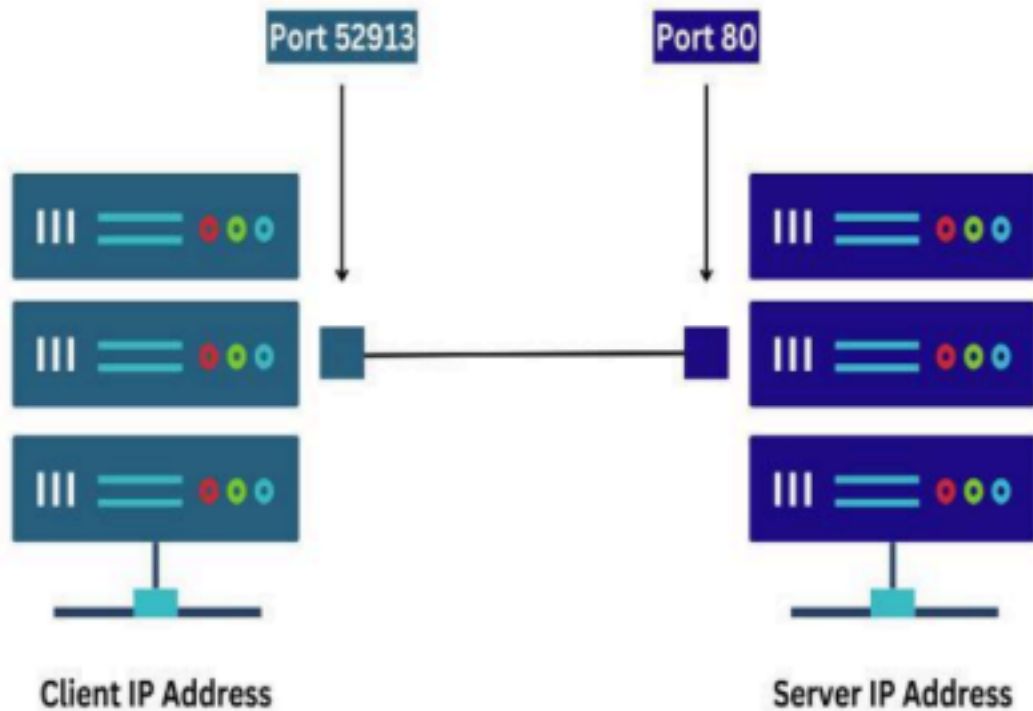
- These are actual hardware connectors on devices.
- Examples include USB for keyboards and storage devices, Ethernet (RJ-45) for

network connections, HDMI for video, and audio ports for sound.

- These are any connector you can plug a device into.

2. Logical/Network Ports

- These are software-based endpoints that let programs communicate over a network.
 - Each port is numbered from 0 to 65535 and is divided into different groups. •
- Well-known Ports (0–1023):** Reserved for standard services like HTTP (80), HTTPS (443), FTP (21), SSH (22).
- **Registered Ports (1024–49151):** Used by specific applications like databases (MySQL 3306, SQL Server 1433).
 - **Dynamic/Private Ports (49152–65535):** Temporarily assigned for client connections, like when your browser opens a port to connect to a website.



Common Port Numbers

Many network services use default port numbers so that communication between clients and servers is standardized and predictable. These commonly used ports allow applications to identify services easily without requiring additional configuration.

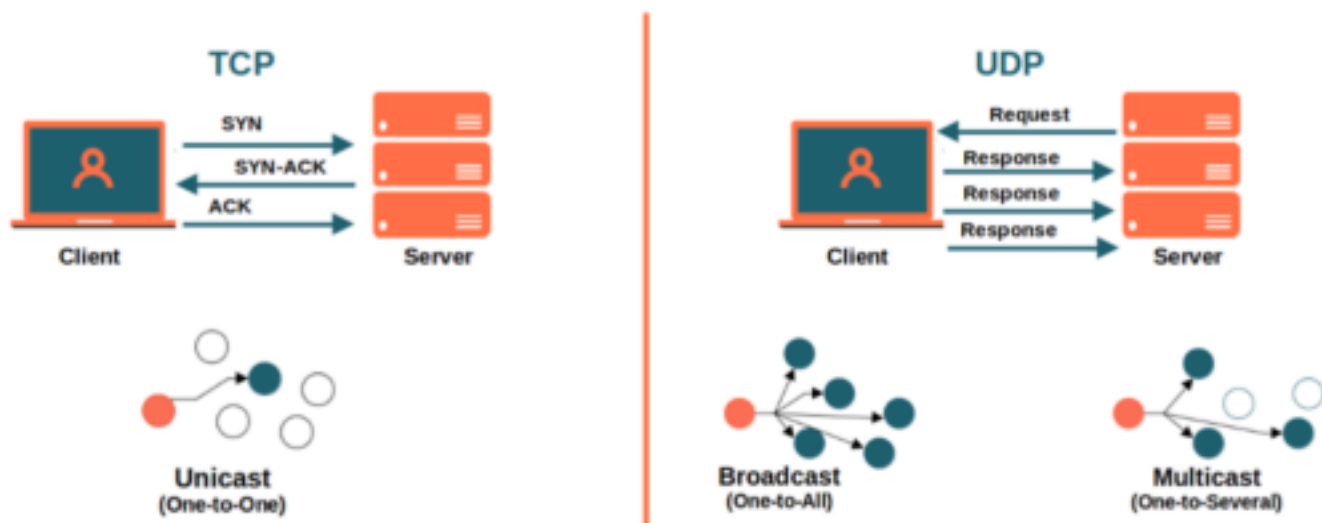
Port Number	Application / Service
20, 21	FTP (File Transfer Protocol)
22	SSH (Secure Shell)
23	Telnet (Terminal Network)
25	SMTP (Simple Mail Transfer Protocol)
53	DNS (Domain Name System)
67 / 68	DHCP (Dynamic Host Configuration Protocol)

80	HTTP (Hypertext Transfer Protocol)
110	POP3 (Post Office Protocol version 3)
143	IMAP (Internet Message Access Protocol)
443	HTTPS (Hypertext Transfer Protocol Secure)
3306	MySQL (My Structured Query Language)
3389	RDP (Remote Desktop Protocol)

Why is UDP faster than TCP?

UDP does not establish a connection, send acknowledgments, or retransmit lost data, reducing overhead and making it ideal for real-time applications like VoIP and online gaming.

Difference Between TCP and UDP at Transport Layer Choosing between TCP and UDP depends on the application's need for reliability versus speed and efficiency.



TCP	UDP
TCP is a connection-oriented protocol	UDP is the connection-less protocol
TCP supports error-checking mechanisms.	UDP has only the basic error-checking mechanism using checksums.
An acknowledgment segment is present.	No acknowledgment segment.
TCP is slower than UDP	UDP is faster, simpler, and more efficient than TCP.
Retransmission of lost packets is possible in TCP, but not in UDP.	There is no retransmission of lost packets in the User Datagram Protocol (UDP)
TCP has a (20-60) bytes variable length header.	The header length is fixed of 8 bytes.